

FONDAMENTI DI PROGRAMMAZIONE WEB

LABORATORIO
Capitolo 7 – Server

Argomenti del giorno

- ▶ Server:
 - ▶ Postman
 - ▶ Express
 - ▶ Creazione Server
 - ▶ Nodemon
 - ▶ Collegamento DB
 - ▶ Routes e Queries

Installazione Postman

► **Windows/Mac/Linux:** <https://www.postman.com/downloads/>

Product ▾ Solutions ▾ Pricing Enterprise Resources ▾

Contact Sales Sign In Sign Up for Free

Download Postman

Download the desktop app for the most complete Postman experience, with native Git support and deeper integration into your development workflow.

The Postman app

Download the app to get started with the Postman API Platform.

[Windows x64](#)

[Download for Windows ARM64](#) →

By downloading and using Postman, I agree to the [Privacy Policy](#) and [Terms](#).

[Release Notes](#) →

Not your OS? Download for Mac ([Intel Chip](#), [Apple Chip](#)) or Linux (x64, arm64)

Impostazioni sulla privacy

Di seguito è riportata una panoramica delle diverse tecnologie che utilizziamo per il trattamento delle informazioni personali.

Essential Opted In ☒

Non è necessario il consenso.

Functional Opted Out ☐

Personalizzazione, moduli compilati automaticamente, ecc.

Analytics Opted Out ☐

Aiutaci a capire come viene utilizzato il nostro sito e come funziona.

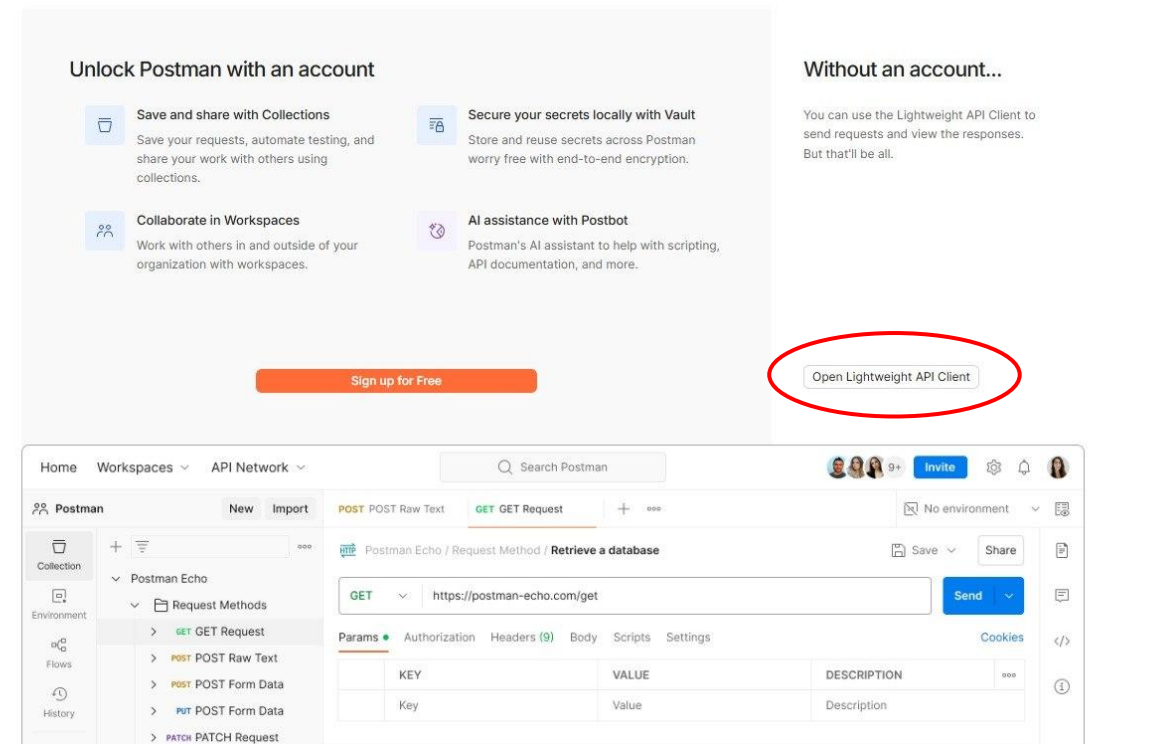
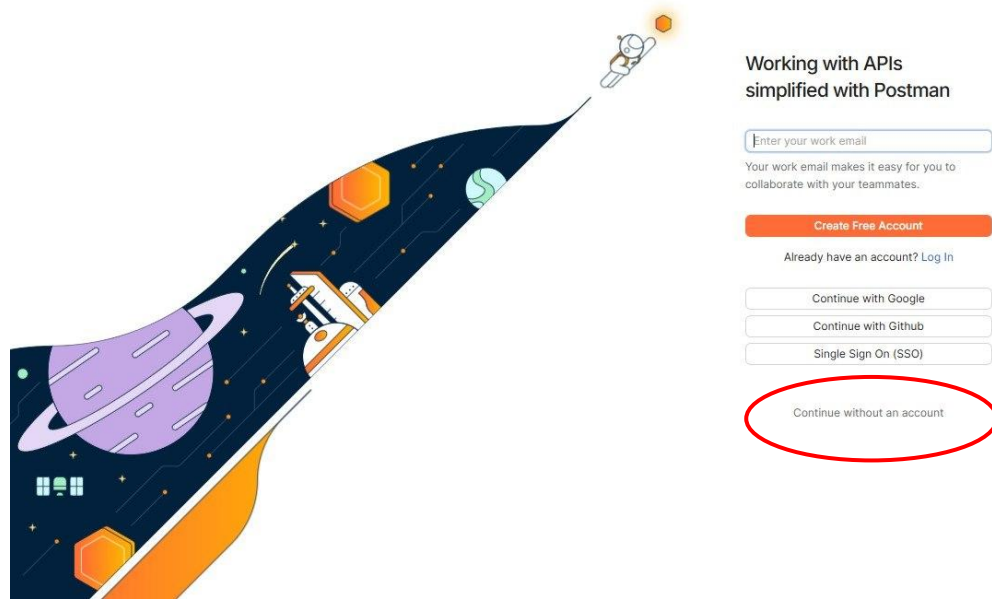
Advertising Opted Out ☐

Aiuta noi e altri a pubblicare annunci pertinenti per te.

[Consulta la nostra politica sulla privacy](#)

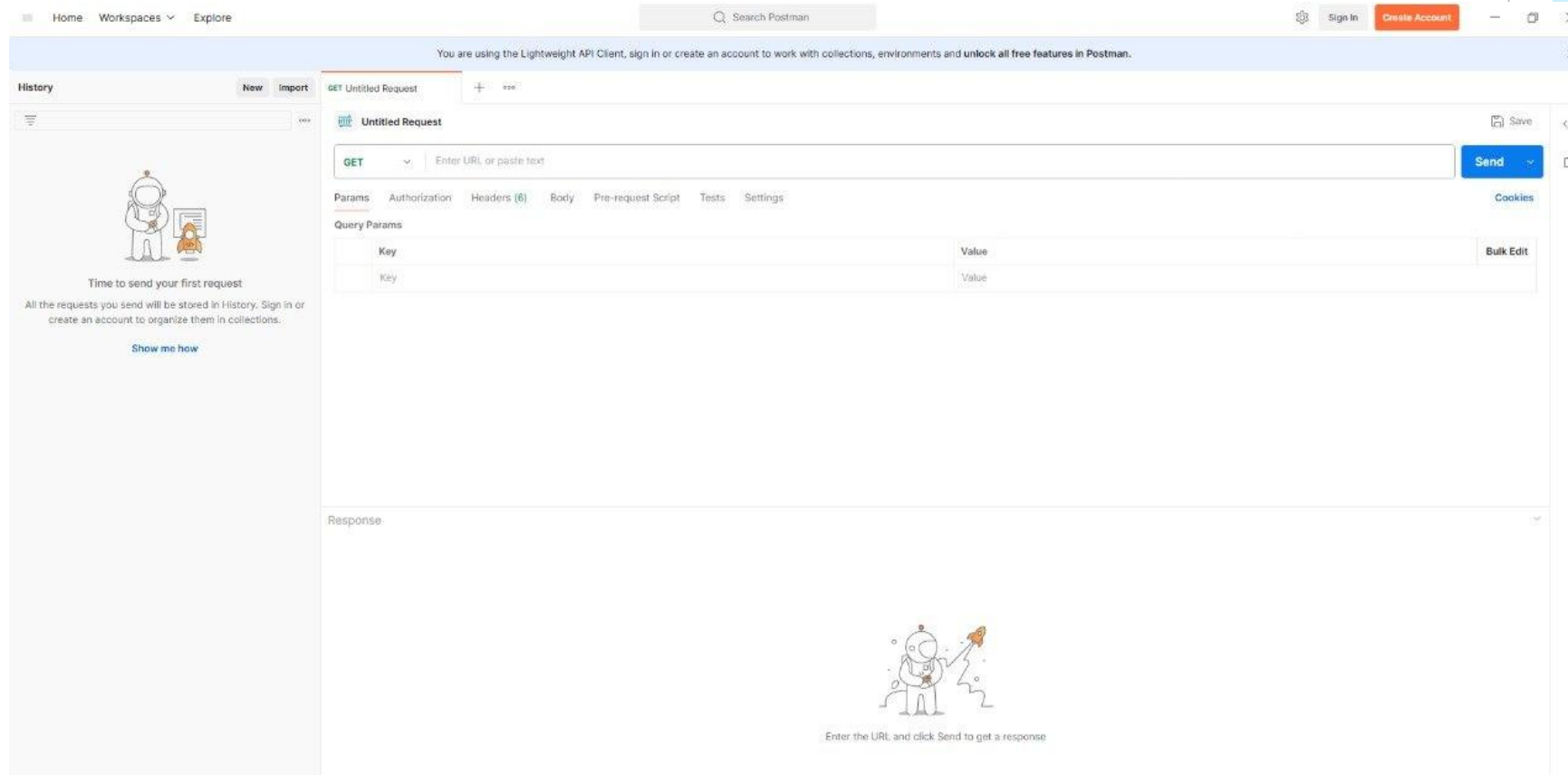
Installazione Postman

- ▶ Vi chiederà di creare un'account. Non è indispensabile, a noi va bene anche la versione base.



Installazione Postman

- Se l'installazione si è conclusa correttamente, vedrete questa schermata.

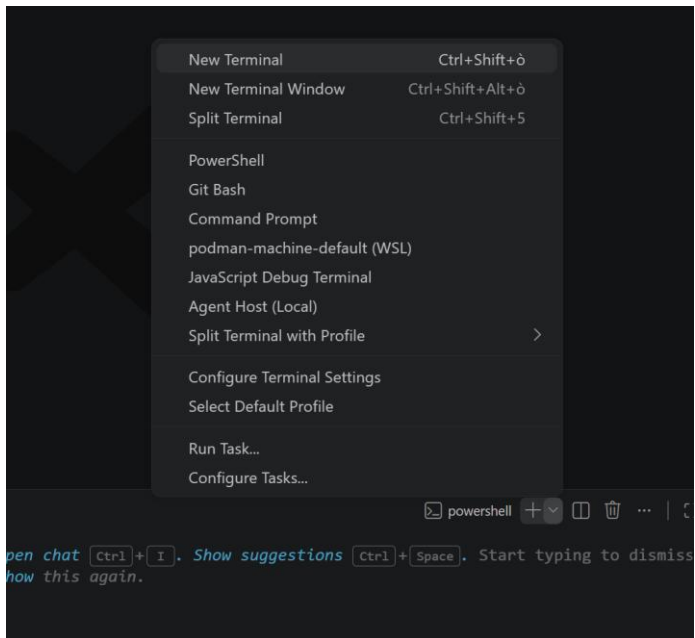


Alternative a Postman

- ▶ Durante le lezioni di laboratorio useremo Postman ma sentitevi liberi di usare software alternativi (Insomnia, Bruno, etc.).

Node.js

- ▶ Entriamo nella cartella del nostro progetto, creiamo una cartella **Server**, apriamo un nuovo terminale dal menu a tendina in alto.



- ▶ Successivamente entriamo nella cartella appena creata e andiamo a creare il nostro **package.json** con il comando **npm init** o **npm init -y** (con l'opzione -y, il terminale non vi chiederà nulla)

```
fabiopili@macbookpro CineVUE2025 % cd Server
fabiopili@macbookpro Server % npm init -y
Wrote to /Users/fabiopili/Code/CineVUE2025/Server/package.json:
```

```
{
  "name": "server",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Node.js – Express

- ▶ Andiamo ad aggiungere Express al nostro package con il comando **`npm install express`** o **`npm i express`**

```
fabiopili@macbookpro Server % npm install express
```

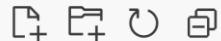
```
added 66 packages, and audited 67 packages in 6s
```

```
14 packages are looking for funding  
run `npm fund` for details
```

```
found 0 vulnerabilities
```

- ▶ Notiamo che nella nostra cartella sono stati creati una nuova cartella **`node_modules`** e due nuovi file **`package.json`** e **`package-lock.json`**:

✓ SERVER



> node_modules

{ } package-lock.json

{ } package.json

- ▶ **`Node_modules`** è la cartella che contiene tutti i package installati localmente per il progetto.
- ▶ **`Package.json`** è il file in cui lo sviluppatore dichiara i gli script disponibili (vedeteli come delle scorciatoie) e i metadata vari. Inoltre, vengono elencati anche i packages necessari (questo viene fatto automaticamente).
- ▶ **`Package-lock.json`** è un registro automatico delle versioni esatte delle dipendenze dei package installati. A differenza del **`package.json`**, in genere questo file viene modificato dai comandi eseguiti.

Node.js – Express

- ▶ Dentro la cartella Server, create un file **server.js**
 - ▶ Useremo questo file per importare **Express**, creare **un'istanza dell'applicazione** e definire alcuni parametri fondamentali, come la **porta** sulla quale il server resterà in ascolto per ricevere le richieste.

server.js

```
const express = require('express');
const app = express();
const port = 3000;

app.listen(port, () => console.log(`app listening
on port ${port}!`));
```

- ▶ Come avviare il server?
 - ▶ Da terminale eseguite il comando **node server.js**

```
PS C:\Users\sandr\Documents\fpw - lez\lezione-7\CineVUE2026\Server> node server.js
app listening on port 3000!
```



Node.js – Package.json

- ▶ Prima di andare avanti, diamo un'occhiata al file **Package.json**
 - ▶ La particolarità di questo file è che oltre ai metadati e alle dipendenze, possiamo definire anche delle shortcut

```
{  
  "name": "server",  
  "version": "1.0.0",  
  "description": "",  
  "main": "server.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1",  
    "start": "node server.js"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs",  
  "dependencies": {  
    "express": "^5.2.1"  
  }  
}
```

- ▶ La particolarità di questo file è che oltre ai metadati e alle dipendenze, possiamo definire anche dei comandi personalizzabili, ovvero delle **scorciatoie**.
- ▶ Definiamo una scorciatoia per avviare il server
 - ▶ In genere lo si associa al comando **start**
- ▶ Ora potrete avviare il server digitando direttamente **npm run start**
 - ▶ I vantaggi di queste scorciatoie saranno più evidenti nei prossimi passi

Node.js – PostgreSQL e Nodemon

- ▶ Ora aggiungiamo i packages di Postgres e Nodemon.
- ▶ Ma a cosa servono?
- ▶ **PostgreSQL** è il package che implementa i metodi che useremo per comunicare con il database creato nella lezione precedente.
 - ▶ *È necessario? Sì, visto che stiamo usando questo RDBMS.*
- ▶ **Nodemon**, invece, è un utility per Node.js che monitora i file che compongono il server e riavvia automaticamente l'applicazione ogni volta che rileva delle modifiche.
 - ▶ *È necessario? Non è indispensabile, ma vi aiuta a velocizzare il lavoro.*

Node.js – PostgreSQL e Nodemon

- ▶ Installate i due package in questo modo:
 - ▶ ***npm i pg-promise*** o ***npm install pg-promise*** per **node-postgres**.
 - ▶ ***npm i nodemon*** o ***npm install nodemon*** per **nodemon**.
- ▶ Quali sono gli effetti di questi due comandi?
 - ▶ Si aggiorna il contenuto della cartella **node-modules** con i file dei due package
 - ▶ Inoltre si aggiorna anche il contenuto dei due file **package.json** e **package-lock.json**, dove troveremo le info dei due pacchetti e della versione utilizzata.
 - ▶ Per chi vuole approfondire, [qui](#) trovate un'estesa descrizione del comando *npm install* e delle varie opzioni disponibili.

Node.js – Come usare Nodemon

- ▶ Abbiamo installato il package Nodemon ma, attualmente, non lo stiamo ancora sfruttando.
- ▶ Un modo per usarlo è modificare il file `package.json`.
- ▶ Nella sezione `scripts`, sostituiamo in `start node server.js` con `nodemon server.js`. Questo permetterà di far modifiche ai file senza avviare continuamente il server.

```
{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1",
    "nodemon": "^3.1.14",
    "pg-promise": "^12.6.2"
  }
}
```

```
PS C:\Users\sandr\Documents\fpw - lez\lezione-7\CineVUE2026\Server> npm run start

> server@1.0.0 start
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
app listening on port 3000!
```

Server

- ▶ Ora definiamo la nostra prima **route** http per l'url di base «/»

server.js

```
const express = require('express');

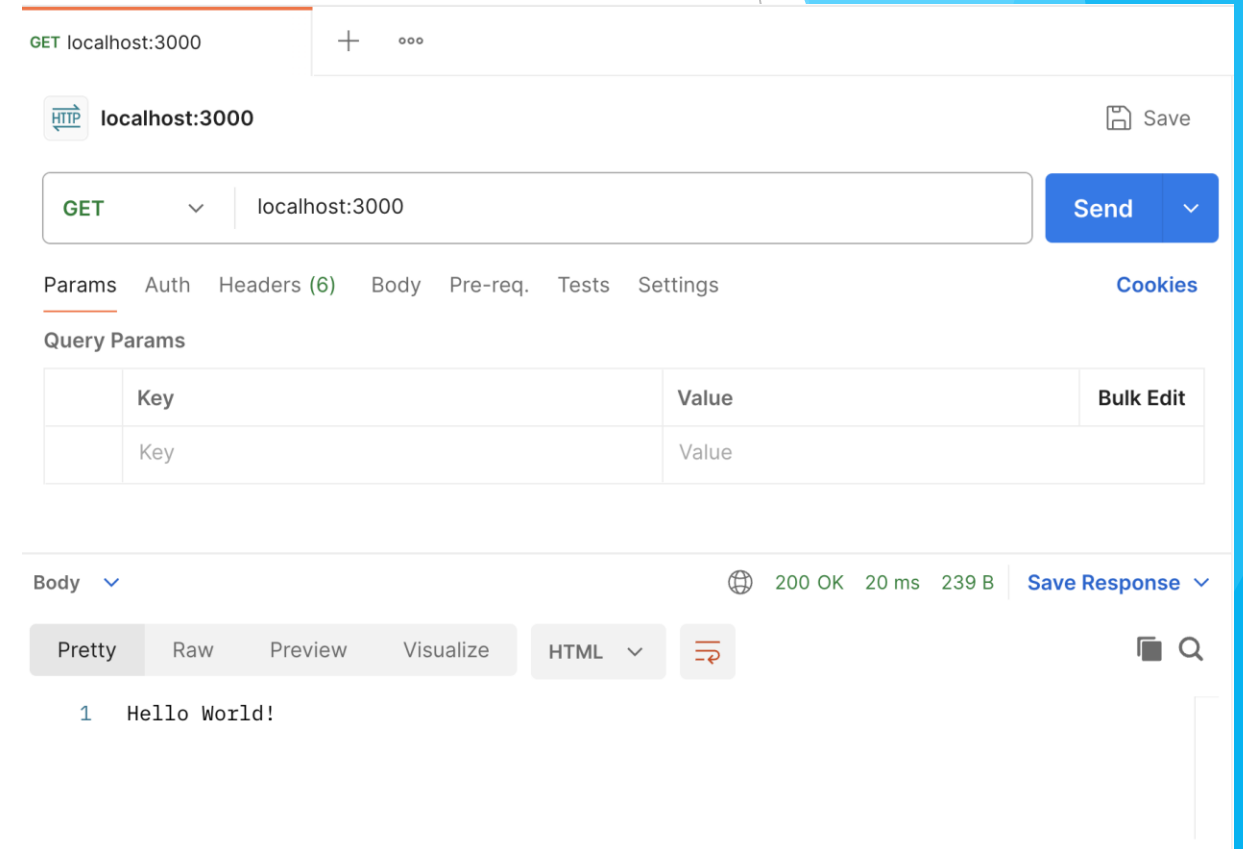
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(port, () => console.log(`app listening on port ${port}!`));
```

Server – Testare una route

- ▶ Aprite **Postman** e definiamo una nuova request
 - ▶ Connettiamoci alla nostra route e vediamo che riceveremo in risposta ciò che abbiamo scritto all'interno dell'oggetto **response**
- ▶ Da adesso useremo Postman come client fittizio per testare le nostre **routes**



Server - DB

- ▶ Ora colleghiamo il server al database **cineva** che abbiamo creato la volta scorsa su **PgAdmin**. Per farlo, creiamo un nuovo file **db.js** nella root del nostro progetto e, sfruttando pg-promise, andiamo a collegarci al nostro db

db.js

```
const pgp = require('pg-promise')();
const db = pgp((
  {
    host: 'localhost',
    port: 5432,
    database: 'cineva',
    user: 'postgres',
    password: 'admin'
  }
));
module.exports = db;
```

- ▶ Ricordatevi che la password è quella che avete impostato durante l'installazione!
- ▶ **module.exports** ci permette di richiamare le funzioni in altri file!

Server

- Andiamo a preparare il nostro server creando una cartella **src**, con all'interno tre sottocartelle, una per ogni tabella del nostro database

> node_modules

▼ src

> film

> recensioni

> utenti

{ } package-lock.json

{ } package.json

JS server.js

Definire una GET

- ▶ All'interno della cartella film creiamo tre file: **film_controller.js**, **film_queries.js** e **film_routes.js**
 - ▶ Perché 3 file? Per tenere separati i vari concetti e massimizzare la manutenibilità del codice.
- ▶ **Routes:** in questo file definiamo tutti gli endpoint che richiameremo in **server.js**.
 - ▶ Associa ogni richiesta HTTP (metodo + PATH) alla funzione del controller che dovrà gestirla.
- ▶ **Controller:** in questo file mettiamo tutti i **controller** che richiameremo in **routes**:
 - ▶ Contiene la logica che collega il client al db. In questo file definiremo i metodi che il router richiamerà per rispondere alle richieste HTTP ricevute.
- ▶ **Queries:** in questo file mettiamo tutte le **queries** che richiameremo dal **controller**.

Definire una GET - Queries

- ▶ Partiamo da **film_queries.js** e definiamo la prima query con cui recuperiamo tutto il contenuto della tabella film.

film_queries.js

```
const getFilms = "SELECT * FROM  
film";  
  
module.exports = {  
  getFilms,  
}
```

Definire una GET - Controller

- ▶ Passiamo al controller. All'interno di **film_controller.js** recuperiamo l'oggetto db dal file di database e creiamo una funzione.
 - ▶ Questa funzione si dovrà collegare al db e, tramite una query, ci deve restituire tutti i film presenti in tabella.
 - ▶ Per farlo, sfrutteremo la query definita nel file **film_queries.js**.

film_controller.js

```
const db = require('../db');
const queries = require('./film_queries');

const getFilmList = async (req, res) => {
  const rows = await db.query(queries.getFilms);
  res.status(200).json(rows);
};

module.exports = {
  getFilmList,
};
```

Definire una GET - Routes

- ▶ Definiamo ora la route. All'interno del file **film_routes.js** richiamiamo l'oggetto Router messo a disposizione da Express.
- ▶ Questo oggetto ci permette di definire diversi **endpoint**, attraverso i quali il client potrà inviare richieste specifiche al server, mantenendo il codice più ordinato, separando le route su più file.
- ▶ Ogni **endpoint** potrà poi essere collegata a una funzione che interroga il database e restituisce una risposta
 - ▶ E sarà qui che useremo le funzioni definite in **film_controller.js**.

film_routes.js

```
const { Router } = require('express');  
const controller = require('./film_controller');  
  
const router = Router();  
  
router.get('/', controller.getFilmList)  
  
module.exports = router;
```

Definire una GET – Server.js

- ▶ L'endpoint che ci permette di recuperare tutti i film presenti nel db è quasi pronto:
 - ▶ Manca solo una cosa, collegare l'oggetto router di **film_routes.js**, e gli endpoint che definisce, con l'oggetto app di **server.js**.
- ▶ Per farlo, prima importiamo l'oggetto contenente le rotte definite nel file **film_routes.js** e, successivamente colleghiamo queste rotte al percorso principale **/film** usando la funzione **use** di app.
 - ▶ In questo modo, ogni richiesta del client che inizia con **/film** verrà gestita dalle rotte presenti in **filmRoutes**.

server.js

```
const express = require('express');
const filmRoutes = require('./src/film/film_routes');

const app = express();
const port = 3000;
app.get('/', (req, res) => {
  res.send('Hello World!');
});
app.use('/film', filmRoutes);
app.listen(port, () => console.log(`app listening on port ${port}!`));
```

Definire una GET – Risultato finale

- ▶ Proviamo a richiamare l'endpoint definito usando POSTMAN
 - ▶ Se avete seguito tutti i passaggi, la chiamata restituirà tutte le righe presenti nella tabella

The screenshot displays the Postman application interface. On the left, the 'History' panel shows a single request: 'GET localhost:3000/film'. The main workspace is divided into several sections. At the top, the request details show 'GET localhost:3000/film' with a 'Send' button. Below this, the 'Params' tab is active, showing a table for 'Query Params' with one row: 'Key' and 'Value'. The 'Body' tab is also visible, showing a JSON response. The status bar at the bottom indicates 'Status: 200 OK', 'Time: 100 ms', and 'Size: 536 B'. A 'Save Response' button is present. The JSON response is displayed in the 'Body' tab, showing a movie record with fields like 'id', 'titolo', 'regista', 'genere', 'descrizione', and 'foto_locandina'.

History

New Import

GET localhost:3000 GET localhost:3000/film + ...

localhost:3000/film

GET localhost:3000/film

Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (7) Test Results

Status: 200 OK Time: 100 ms Size: 536 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "titolo": "Focus -Niente è come sembra",
4   "regista": "Glenn Ficarra",
5   "genere": "commedia",
6   "descrizione": "Un esperto truffatore, Nicky Spurgeon, cresciuto in una famiglia di truffatori, \nha una propria \n\"azienda\" con cui
7     progetta e mette in atto molteplici colpi",
8   "foto_locandina": "focus.png"
9 }
10
```

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

Definire una GET con parametro

- ▶ Creiamo una nuova query che ci permetta di selezionare tutti i dati di un film
 - ▶ Il processo è come quello della query vista prima, in questo caso però passiamo anche un parametro con la dicitura **:id**

film_queries.js

```
const getFilms = "SELECT * FROM film";
const getFilmById = "SELECT * FROM film
WHERE id = $1";

module.exports = {
  getFilms,
  getFilmById,
}
```

film_routes.js

```
router.get('/', controller.getFilmList)
router.get('/:id', controller.getFilmById)

module.exports = router;
```

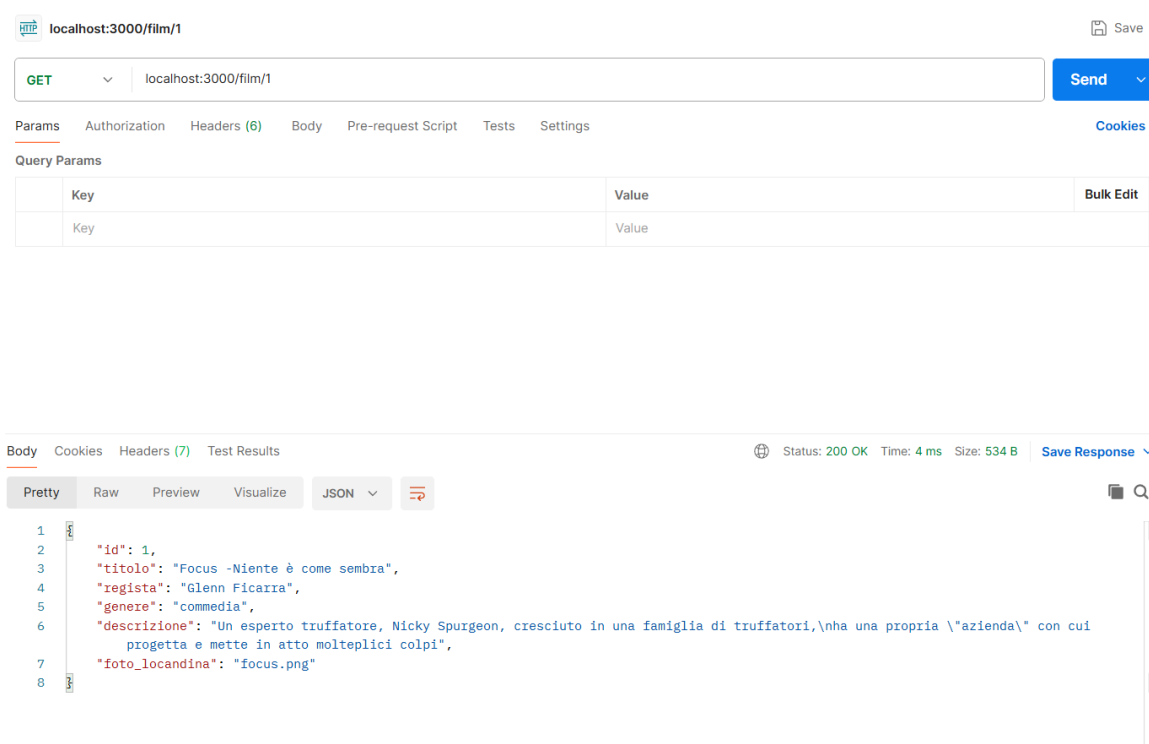
film_controller.js

```
const getFilmById = async (req, res) => {
  const { id } = req.params;
  const film = await db.oneOrNone(queries.getFilmById, [id]);
  res.status(200).json(film);
};

module.exports = {
  getFilmList,
  getFilmById,
};
```


Definire una GET con parametro

- ▶ Proviamo a richiamare l'endpoint definito usando POSTMAN passando il parametro id direttamente dall'url



localhost:3000/film/1

GET localhost:3000/film/1

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

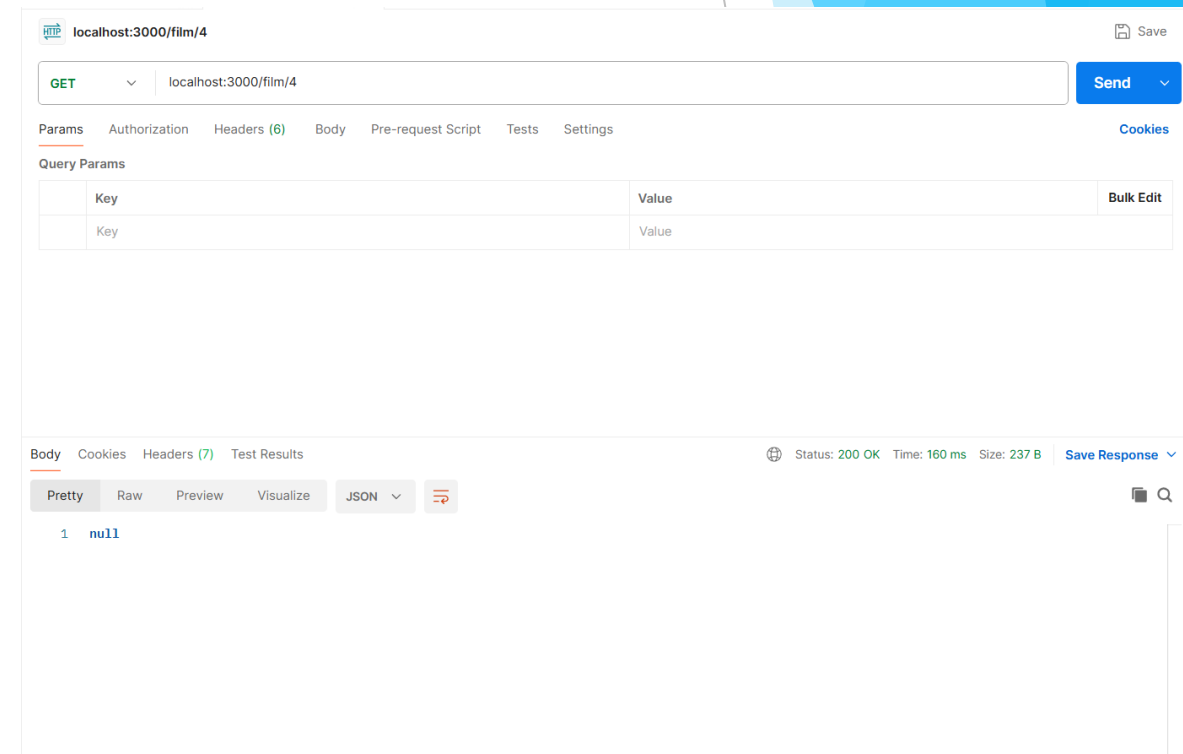
Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (7) Test Results

Status: 200 OK Time: 4 ms Size: 534 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "titolo": "Focus -Niente è come sembra",
4   "regista": "Glenn Ficarra",
5   "genere": "commedia",
6   "descrizione": "Un esperto truffatore, Nicky Spurgeon, cresciuto in una famiglia di truffatori,\\nha una propria \\\"azienda\\\" con cui
7   progetta e mette in atto molteplici colpi",
8   "foto_locandina": "focus.png"
}
```



localhost:3000/film/4

GET localhost:3000/film/4

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (7) Test Results

Status: 200 OK Time: 160 ms Size: 237 B Save Response

Pretty Raw Preview Visualize JSON

```
1 null
```

Definire una POST

- ▶ Ora che sappiamo come richiamare dati dal db, proviamo a **inserire** un nuovo film.
- ▶ In **film_queries.js** creiamo la query con cui aggiungere un film

film_queries.js

```
const getFilms = "SELECT * FROM film";
const getFilmById = "SELECT * FROM film WHERE id = $1";
const addFilm = "INSERT INTO film (id, titolo, regista, genere, descrizione,
foto_locandina) VALUES ($1, $2, $3, $4, $5, $6)";

module.exports = {
  getFilms,
  getFilmById,
  addFilm,
}
```

Definire una POST

- ▶ In **film_controller.js** creiamo la nostra funzione **addFilm**.
- ▶ Definiamo un oggetto con tutti gli attributi di un film ricevuti in input. Più precisamente, questo oggetto contiene le informazioni presenti nel body della richiesta (req) fatta dal client al server.
- ▶ Prima di inserire un film dobbiamo verificare che non sia già presente nel db
 - ▶ Possiamo farlo tramite `getFilmById`
- ▶ In caso affermativo, verrà restituito il messaggio «Il film esiste già nel db» con il codice 409. Altrimenti possiamo proseguire con l'invio del nostro oggetto al db.
 - ▶ Usiamo **none** perché la insert con **pg-promise** non restituisce nulla se va a buon fine.

film_controller.js

```
const addFilm = async (req, res) => {
  const {id, titolo, direttore, genere, descrizione, foto_locandina} = req.body;

  const existing = await db.oneOrNone(queries.getFilmById, [id]);
  if (existing) {
    return res.status(409).json({ error: 'Il film esiste già nel database' });
  }

  await db.none(queries.addFilm, [id, titolo, direttore, genere, descrizione,
    foto_locandina]);
  res.status(201).json({ message: 'Film aggiunto con successo' });
}

module.exports = {
  getFilmList,
  getFilmById,
  addFilm,
};
```

Queries – POST

- In **film_routes.js** aggiungiamo una **post** che avrà come route **"/"** proprio come la nostra **get**

film_routes.js

```
const { Router } = require('express');
const controller = require('./film_controller');

const router = Router();

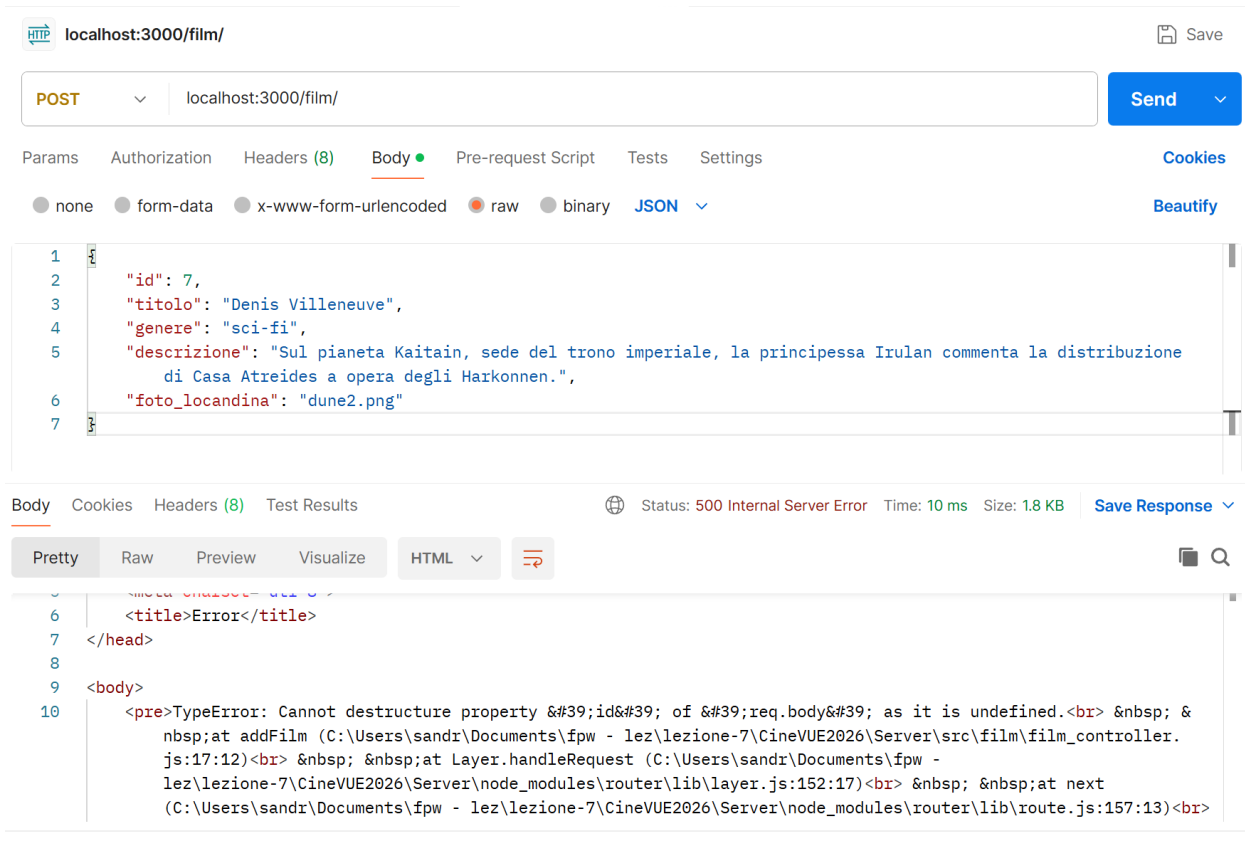
// GET
router.get('/', controller.getFilmList)
router.get('/:id', controller.getFilmById)

// POST
router.post('/', controller.addFilm)

module.exports = router;
```

Queries – POST

- ▶ Andiamo a testarlo su Postman



- ▶ **ERRORE!**
- ▶ In questo momento Express sta processando un JSON, il body della nostra request, senza sapere che si tratta di un JSON!

Queries – POST

- ▶ Su server.js andiamo ad includere **express.json()**
- ▶ **Express.json()** è il nostro middleware per il parsing del body, senza di questo infatti la nostra request sarà undefined!

server.js

```
const express = require('express');
const filmRoutes = require('./src/film/film_routes');

const app = express();
app.use(express.json());
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.use('/film', filmRoutes);
app.listen(port, () => console.log(`app listening on port ${port}!`));
```

Queries – POST

- Andiamo a testarlo ora su Postman

The screenshot shows the Postman interface for a POST request to `localhost:3000/film/`. The request body is a JSON object with the following fields:

```
1 {
2   "id": 7,
3   "titolo": "Denis Villeneuve",
4   "genere": "sci-fi",
5   "descrizione": "Sul pianeta Kaitain, sede del trono imperiale, la principessa Irulan commenta la distribuzione di Casa Atreides a opera degli Harkonnen.",
6   "foto_locandina": "dune2.png"
7 }
```

The response is displayed in the bottom panel, showing a status of 201 Created, a time of 223 ms, and a size of 280 B. The response body is a JSON object:

```
1 {
2   "message": "Film aggiunto con successo"
3 }
```

The screenshot shows the Postman interface for a POST request to `localhost:3000/film/`. The request body is the same JSON object as in the previous screenshot:

```
1 {
2   "id": 7,
3   "titolo": "Denis Villeneuve",
4   "genere": "sci-fi",
5   "descrizione": "Sul pianeta Kaitain, sede del trono imperiale, la principessa Irulan commenta la distribuzione di Casa Atreides a opera degli Harkonnen.",
6   "foto_locandina": "dune2.png"
7 }
```

The response is displayed in the bottom panel, showing a status of 409 Conflict, a time of 57 ms, and a size of 285 B. The response body is a JSON object:

```
1 {
2   "error": "Il film esiste già nel database"
3 }
```

Esercizi

- ▶ Creare i rispettivi file controller, queries e routes per le classi utenti e recensioni
- ▶ Creare le funzioni analoghe a quelle viste a lezione per ogni classe
 - ▶ Quando aggiungete una nuova recensione controllate che utente e film esistano prima di inserire una nuova recensione
- ▶ Modificate la addFilm in modo che il controllo sull'esistenza del film non avvenga più sull'id ma su titolo e regista insieme